

IT342 Phase 1 – User Registration and Login

Project: Knock Knock

Student: Niña Nicole S. Villadarez

Course: IT342

1. GitHub Repository Link

Repository Link: https://github.com/theninanicole/IT342_KnockKnock_G4_Villadarez

2. Final Commit for Phase 1

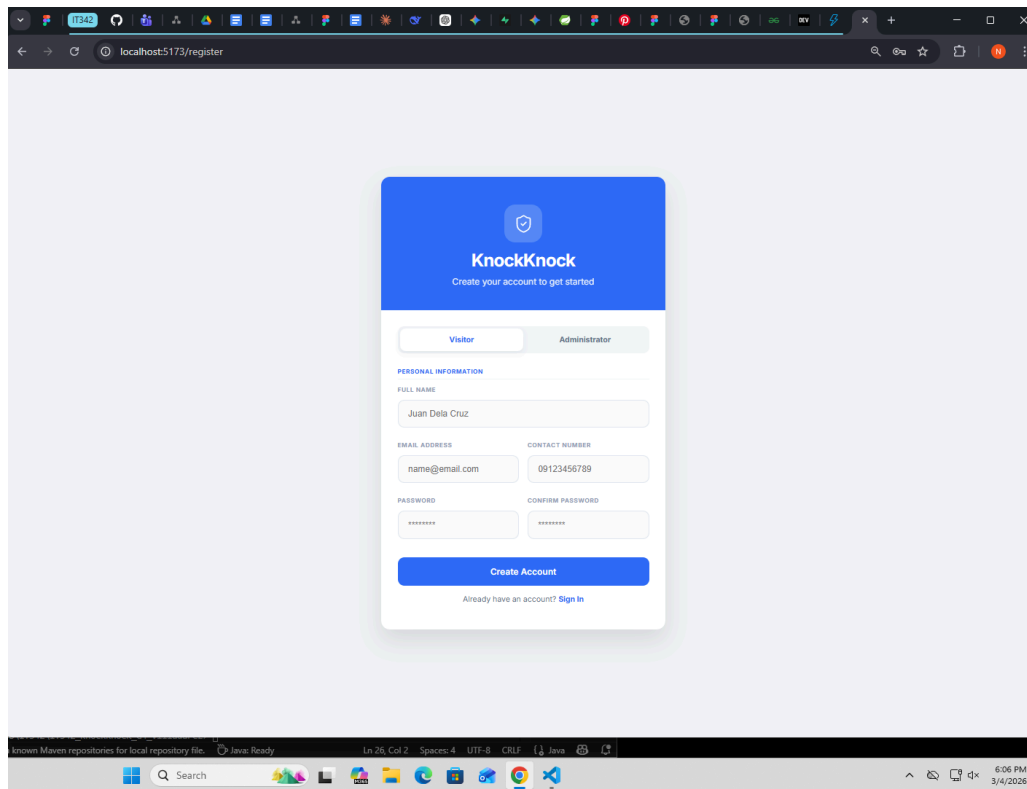
Final commit message used in GitHub: *IT342 Phase 1 – User Registration and Login Completed*

Commit Link / Hash:

https://github.com/theninanicole/IT342_KnockKnock_G4_Villadarez/commit/130c06a4d7cf95a30aaa7546fd13ddac0de1ff99

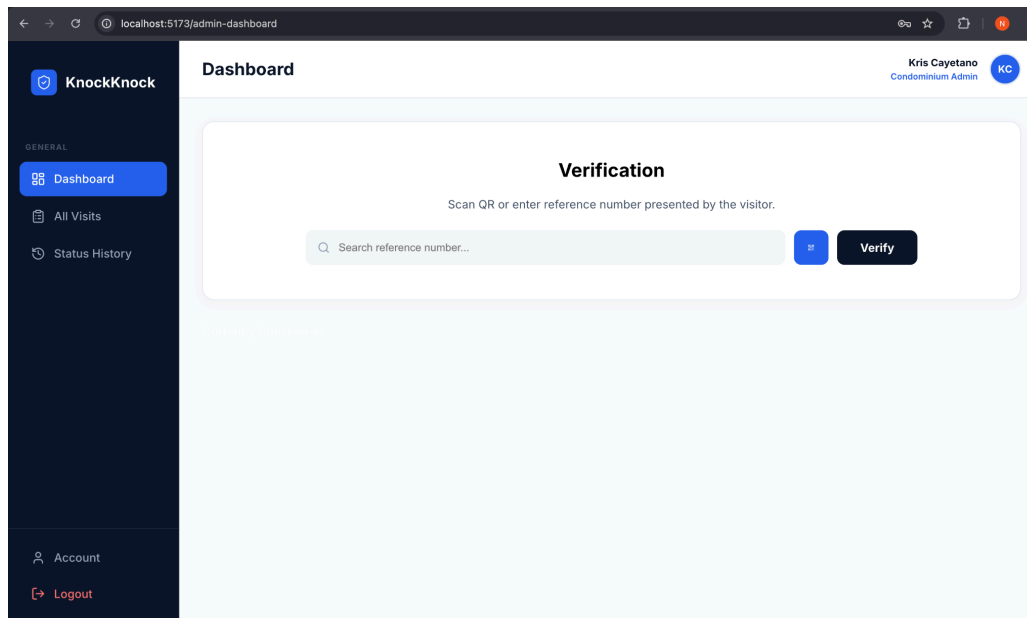
3. Screenshots of Implementation

Register Page



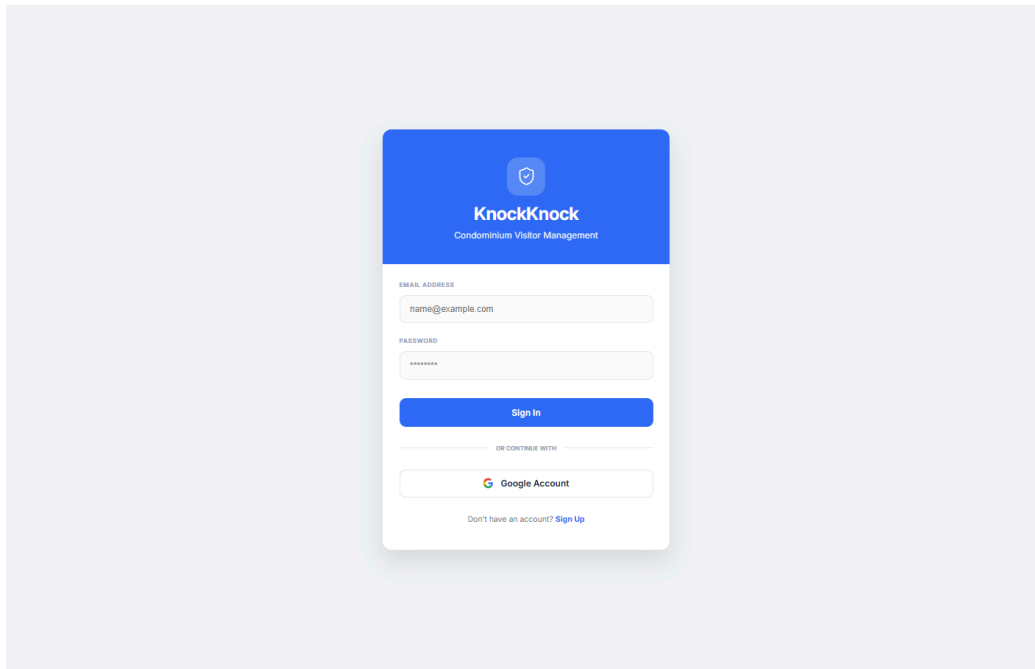
The screenshot shows a web browser window with the address bar displaying 'localhost:5173/register'. The page features a central registration form with a blue header containing the 'KnockKnock' logo and the text 'Create your account to get started'. Below the header, there are two tabs: 'Visitor' (selected) and 'Administrator'. The form is titled 'PERSONAL INFORMATION' and includes fields for 'FULL NAME' (filled with 'Juan Dela Cruz'), 'EMAIL ADDRESS' (filled with 'name@email.com'), and 'CONTACT NUMBER' (filled with '09123456789'). There are also fields for 'PASSWORD' and 'CONFIRM PASSWORD', both masked with 'xxxxxx'. A blue 'Create Account' button is at the bottom, with a link 'Already have an account? Sign In' below it. The browser's taskbar at the bottom shows the Windows logo, a search bar, and various application icons, with the system clock indicating 6:06 PM on 3/4/2026.

Successful user registration

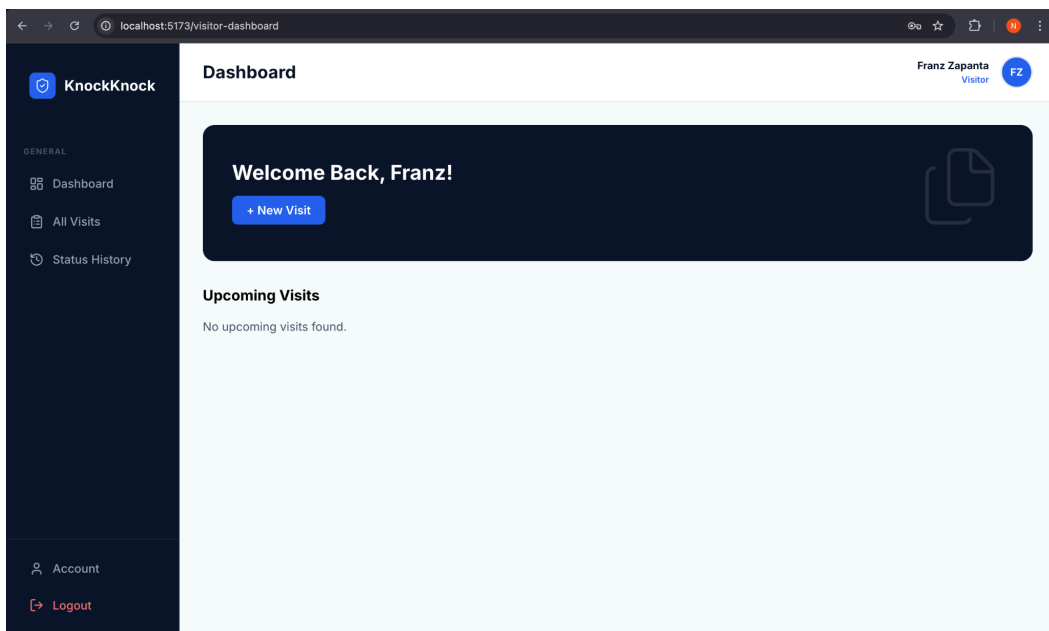


The screenshot shows the 'KnockKnock' admin dashboard. The left sidebar contains a 'GENERAL' section with 'Dashboard' (selected), 'All Visits', and 'Status History'. Below this is an 'Account' section with 'Logout'. The main content area is titled 'Dashboard' and features a 'Verification' section. This section includes the text 'Scan QR or enter reference number presented by the visitor.' and a search bar with the placeholder 'Search reference number...'. A blue 'qr' icon and a dark blue 'Verify' button are next to the search bar. The top right of the dashboard shows the user 'Kris Cayetano' with the role 'Condominium Admin' and a profile icon labeled 'KC'. The browser's address bar shows 'localhost:5173/admin-dashboard'.

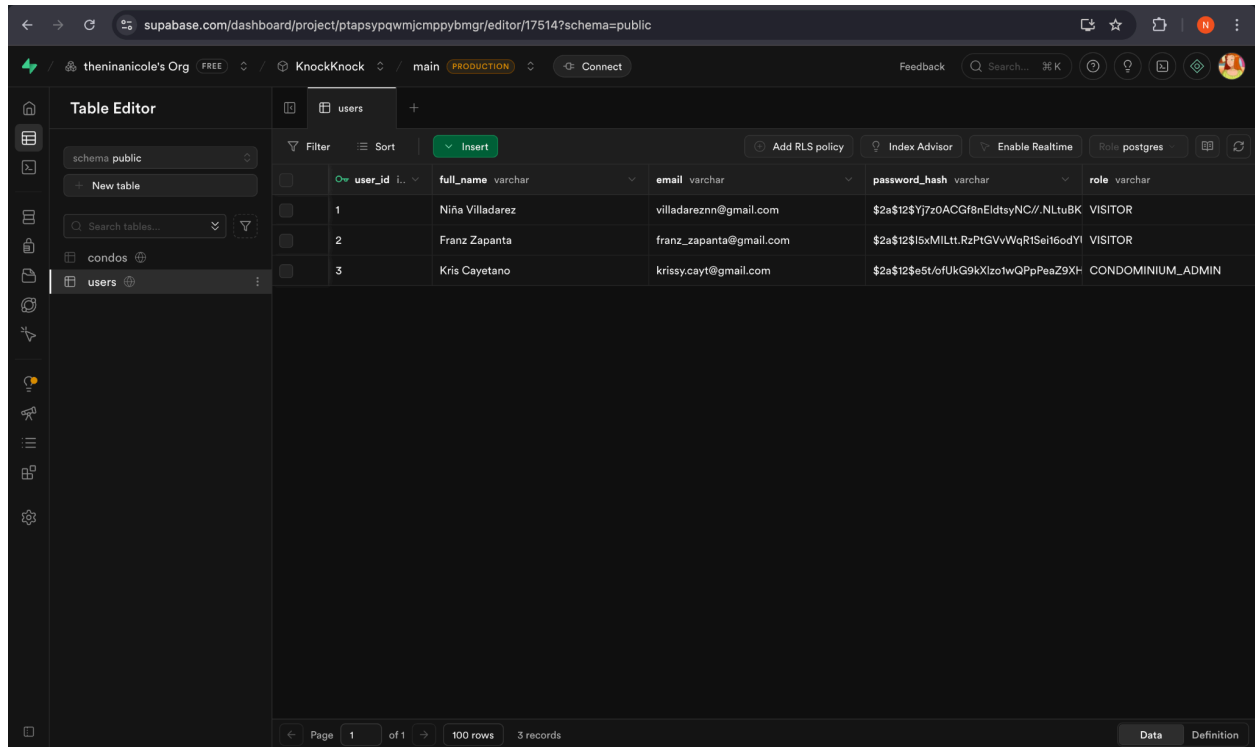
Login Page



Successful user login



Database Record Showing Registered User



The screenshot shows the Supabase Table Editor interface for the 'users' table in the 'public' schema. The table has five columns: 'user_id', 'full_name', 'email', 'password_hash', and 'role'. There are three records in the table. The interface includes a sidebar with a table list, a top navigation bar with project information, and a bottom status bar showing 'Page 1 of 1' and '100 rows'.

	user_id	full_name	email	password_hash	role
1		Niña Villadarez	villadareznn@gmail.com	\$2a\$12\$Yj7z0ACGf8nEldsyNC//NLtuBK	VISITOR
2		Franz Zapanta	franz_zapanta@gmail.com	\$2a\$12\$I5xMILTt.RzPtGVvWqRtSei6odYI	VISITOR
3		Kris Cayetano	krissy.cayt@gmail.com	\$2a\$12\$eSt/fUkG9KXlzo1wQPpPeaZ9XH	CONDOMINIUM_ADMIN

4. Implementation Summary

User Registration

The system allows users to create an account through the registration page. The registration form collects the necessary user information and sends it to the backend server for processing.

Registration Fields Used:

- Full Name
- Email Address
- Contact Number
- Password
- Confirm Password

For administrators registering a condominium, additional fields are included:

- Condominium Name
- Condominium Address

➤ Validation Process

- The system performs several validation checks before creating a user account:
- Required field validation ensures all mandatory fields are filled.
- Email format validation checks that the email address is valid.
- Password confirmation validation ensures the password and confirm password fields match.
- Backend validation verifies the data before storing it in the database.
- If validation fails, the system displays an appropriate error message.

➤ Preventing Duplicate Accounts

- Duplicate accounts are prevented by checking if the email already exists in the database before creating a new user.
- The backend uses a repository query:
- `existsByEmail(email)`
- If the email already exists, registration is rejected and the user is informed that the account already exists.

➤ Secure Password Storage

- Passwords are not stored as plain text. Instead, they are securely stored using password hashing.
- The system uses Spring Security password encryption to hash the password before saving it in the database.
- This prevents attackers from reading user passwords even if the database is compromised.

User Login

The login system allows registered users to access the application by verifying their credentials.

Login Credentials Used:

- Email address
- Password

User Verification Process

When a user submits the login form:

- The frontend sends a login request to the backend API.
- The backend searches for the user using the provided email.
- The system compares the entered password with the stored encrypted password.
- If the credentials match, the user is authenticated successfully.

After Successful Login

The system identifies the user role.

The user is redirected to the appropriate dashboard.

Example:

Visitor → Visitor Dashboard

Admin → Admin Dashboard

This ensures users access features based on their role.

Database Table

The system uses a *users* table to store registered user accounts.

Table: users

user_id	Unique identifier for each user
full_name	User's full name
contact_number	User's contact number
email	Unique email address
password_hash	Encrypted password
role	User role (VISITOR or CONDOMINIUM_ADMIN)
created_at	Timestamp when account was created
updated_at	Timestamp when account was updated

This table stores all user account information used for authentication and system access.

API Endpoints

The backend provides REST API endpoints for authentication.

User Registration:

POST /api/auth/register/visitor

POST /api/auth/register/condo-admin

These endpoints create a new user account in the system.

User Login

POST /api/auth/login

This endpoint verifies the user credentials and returns the authenticated user information.